Loup M. Marquez Luttringer – Project Manager

Matheus Lopez – Back-End & Front-End Developer

Noah Gibelli – AI Developer

Version: v1.0

Date: October 29, 2025

Horus Project – Security Implementation Justification

(Valorizing Security, Why Security is Critical in Multi-Agent Systems, Choosing the Right Security Technologies, Auditing and Compliance & Future Work and Improvements)

Point 1 – A – Valorizing Security

When we started to build the Horus Orchestrator, we knew that security could not be an optional feature. Since the system was designed to be self-hosted and used to manage sensitive data, we had to guarantee that only authorized users and agents could access or manipulate information.

Security in Horus is not a simple authentication step — it's a complete access control architecture, based on modern principles such as *Role-Based Access Control (RBAC)*, *JWT token authentication*, and *event auditing*.

Our main goal was to make Horus secure by design, meaning that every interaction between a user, an agent, or the orchestrator must be verified, logged, and traceable. Like inside an organization, each role has specific rights. We can have the user role that just requires agents, or we can have the administrator role that can add or delete agents and also can delete users. Our wish was to stay traditional from a certain point of view but also to have an efficient and modern security process. On this way, each JWT token contain the information on each user. So, each function can control rights of the user, information details, …

Each event is traceable. On this way, we can save time to fix problems or also detected security issues. Each log is stocked inside a persistent database that can allow us to compute some metrics that can allow us to trace the efficiency of our program.

Point 2 – B – Why Security is Critical in Multi-Agent Systems

Multi-agent AI systems are by definition distributed and autonomous. Each agent can process data, make decisions, and communicate with others. While this offers flexibility and scalability, it also increases the attack surface — every new agent or connection is a potential security risk.

Without a solid access control model, a compromised agent could expose sensitive company data or trigger unintended workflows. That's why the orchestrator acts as a security gatekeeper: it validates identities, controls roles, and ensures that each agent only performs authorized actions.

In Horus, security is not centralized only for convenience — it's centralized for control and visibility. This process can give a big flexibility to users to design their proper security rules without impacting code implementation. Also, it can allow us to maintain full auditability of all interactions.

Using Redis cache instead of cookies was a wish from us to enforce as much as possible our security level. Cookies are often used in small projects to remember, for example when a user logs in. But for a project that you want to deploy in a company, it can be very less efficient. And it can allow us to have a modular code. Each container and part of the code are dissociated. It allows us to have also a more speed solution. The API doesn't have to manage cookies and store this information so finally the response time is faster because we have removed overkill part like this one. In the future, our wish is to have a big security base like Oauth or OpenID. Having for example the ability to use google to have an authentication process can enforce our security barrier, obtaining the help of big infrastructure to have more complex token. Instead of Capability-based Security, that is more complex to implement and less friendly in comparison with the RBAC process. For the moment having an administrator and users is sufficient. Our features are not enough complex, and our security implementation follows the architecture plan that we made at the start of the project. We selected JWT and RBAC because they provide the right balance between simplicity, scalability, and control, especially for an on-premises environment where we want to avoid any external authentication service. Each security implementation is not just here because we want to collect as many securities process as possible. We try to think about the need related to a selected security process, thinking about if it's overkill or not for our program or if it's really important to implement it. We try to find an answer to a need that we can have behind a security process.

Point 3 – C – Choosing the Right Security Technologies

To achieve our security goals, we selected a combination of proven, open-source technologies that are compatible with our Python-based ecosystem.

1. JWT (JSON Web Token) Authentication
    - Each user receives a signed token at login, containing their ID and role.
    - The token is validated on every request to the orchestrator.
    - This ensures stateless authentication — scalable, simple, and secure.
2. RBAC (Role-Based Access Control)
    - We defined two initial roles in Phase 1:
        - Admin: can manage users, agents, and configurations.
        - User: can access and interact with the orchestrator through the dashboard.
    - Access control is enforced at the route level with FastAPI dependencies, guaranteeing that restricted actions cannot be performed by unprivileged users.
3. Hybrid Logging and Audit Trails
    - Every important event (login, logout, agent registration, errors, agent request) is recorded in the CriticalLogs database table.
    - This makes the entire system traceable and helps administrators investigate problems or suspicious activity.
4. Redis Session Management
    - Redis stores temporary sessions and user tokens.
    - When a user logs out, their session key is deleted, ensuring that no old tokens remain valid.
    - Each session has a time-to-live (TTL), automatically expiring after one hour to prevent misuse.
    - Each token inside the database stays for 2 hours to be sure to optimize the storage of the container.
5. Encrypted Communication and Isolation
    - All services (orchestrator, agents, and dashboard) communicate within isolated Docker networks.
    - No public access is allowed by default of the admin, only the admin can expose ports explicitly.
    - Each system variable is crypted inside the code to prevent hacking attacks.

Point 4 – D – Auditing and Compliance

Security is not only about access control — it's also about accountability. That's why Horus integrates a hybrid logging system that creates a transparent record of every action performed inside the platform.

Each event is categorized (SYSTEM, SECURITY, AGENT, USER, DATABASE) and timestamped. Logs can be viewed in real time or stored in the database for long-term compliance reporting.

On this way, we can prepare our solution for security certifications like also TLS, or also HTTPS.

Point 5 – E – Future Work and Improvements

While the current implementation ensures strong protection, we have already identified future improvements to extend our security capabilities:

- Fine-Grained RBAC Policies: Add more roles (developer, auditor, analyst) with specific permissions for actions and data access.
- Token Rotation and Revocation: Automatically renew tokens and invalidate them after inactivity.
- WebSocket Authentication: Implement role validation and origin checks for real-time connections.
- Security Monitoring Dashboard: Visualize log events, failed authentications, and agent heartbeats.

These improvements will move Horus from a secure infrastructure to a fully governed AI ecosystem. They still are wishes because we have to think about the need behind.